

Pyxis: Scheduling Mixed Tasks in Disaggregated Datacenters

Sheng Qi, Chao Jin, Mosharaf Chowdhury *Member, IEEE*, Zhenming Liu, Xuanzhe Liu, *Senior Member, IEEE*, Xin Jin, *Senior Member, IEEE*

Abstract—Disaggregating compute from storage is an emerging trend in cloud computing. Effectively utilizing resources in both compute and storage pool is the key to high performance. The state-of-the-art scheduler provides optimal scheduling decisions for workloads with homogeneous tasks. However, cloud applications often generate a mix of tasks with diverse compute and IO characteristics, resulting in sub-optimal performance for existing solutions.

We present Pyxis, a system that provides optimal scheduling decisions for mixed workloads in disaggregated datacenters with theoretical guarantees. Pyxis is capable of maximizing overall throughput while meeting latency SLOs. Pyxis decouples the scheduling of different tasks. Our insight is that the optimal solution has an “all-or-nothing” structure that can be captured by a single *turning point* in the spectrum of tasks. Based on task characteristics, the turning point partitions the tasks either all to storage nodes or all to compute nodes (none to storage nodes). We theoretically prove that the optimal solution has such a structure, and design an online algorithm with sub-second convergence. We implement a prototype of Pyxis. Experiments on CloudLab with various synthetic and application workloads show that Pyxis improves the throughput by 3–21× over the state-of-the-art solution.

Index Terms—Task scheduling, disaggregated datacenter, resource allocation.

APPENDIX

PARETO OPTIMALITY OF (ρ_C, ρ_S)

We have the following theorem about the relationship between ρ_C and ρ_S .

Theorem A.1. *Let R be the optimal throughput under SLO constraints. There exists a split ratio vector $\vec{x}$ s.t. the corresponding load pair $(\rho_C(R, \vec{x}), \rho_S(R, \vec{x}))$ is Pareto Optimal.*

Let $\vec{x}$ be an optimal split ratio vector $\vec{x}$ where $(\rho_C(R, \vec{x}), \rho_S(R, \vec{x}))$ is not Pareto Optimal, i.e., there exists another split ratio vector $\vec{y}$ where $\rho_C(R, \vec{y}) \leq \rho_C(R, \vec{x})$ and $\rho_S(R, \vec{y}) \leq \rho_S(R, \vec{x})$. We prove the theorem by showing that $\vec{y}$ is also feasible, i.e., does not violate the SLO constraints in equation (9) and (10). This is because the latency SLO metrics T_i^C and T_i^S are monotonic in ρ_C and ρ_S , respectively, and none of ρ_C and ρ_S under $\vec{y}$ exceeds the original load under

$\vec{x}$. Consequently, $\vec{y}$ also yields the optimal throughput R . In this way, we may transform any non-Pareto-Optimal solution into a Pareto Optimal one, concluding the proof.

PROOF OF THEOREM IV.1

Let $\vec{x}$ be an optimal split ratio vector. We say a pair $i < j$ is a *bad* pair if $x_i < 1$ and $x_j > 0$. We show that when the number of bad pairs is non-zero, we can always build another optimal solution with a reduced number of bad pairs, i.e., there exists an optimal solution with no bad pairs. Specifically, let $i < j$ be an arbitrary bad pair, we define

$$\delta = \sup_{\delta} \left\{ x_i + \frac{\delta}{p_i c_i} \leq 1 \wedge x_j - \frac{\delta}{p_j c_j} = 0 \right\} \quad (1)$$

We then construct a new split ratio vector $\vec{y}$ with overall throughput R unchanged and

$$y_k = x_k \quad \text{for } k \neq i, j \quad (2)$$

$$y_i = x_i + \frac{\delta}{p_i c_i} \quad (3)$$

$$y_j = x_j - \frac{\delta}{p_j c_j} \quad (4)$$

Note that either $y_i = 1$ or $y_j = 0$ by the definition of δ , so the new solution eliminates one bad pair. It remains to show that *vecy* is still feasible. Similar to our proof of Theorem A.1, it suffices to show that none of the new ρ_C and ρ_S exceeds the original load. Bring $\vec{x}$ and $\vec{y}$ into the formula of ρ_C and ρ_S , we first have

$$\begin{aligned} & (1 - y_i)p_i c_i + (1 - y_j)p_j c_j \\ &= \left(1 - \left(x_i + \frac{\delta}{p_i c_i} \right) \right) p_i c_i + \left(1 - \left(x_j - \frac{\delta}{p_j c_j} \right) \right) p_j c_j \\ &= (1 - x_i)p_i c_i + (1 - x_j)p_j c_j \end{aligned}$$

So ρ_C remains unchanged. And for ρ_S

$$\begin{aligned} & y_i p_i s_i + y_j p_j s_j \\ &= \left(x_i + \frac{\delta}{p_i c_i} \right) p_i s_i + \left(x_j - \frac{\delta}{p_j c_j} \right) p_j s_j \\ &= x_i p_i s_i + x_j p_j s_j + \delta \left(\frac{s_i}{c_i} - \frac{s_j}{c_j} \right) \\ &\leq x_i p_i s_i + x_j p_j s_j \end{aligned}$$

where the last inequality holds because we sort s_i/c_i in ascending order. Therefore, the new ρ_S under $\vec{y}$ is lower.

Finally, we show that when there is no bad pairs, the solution has an “all-or-nothing” structure. Let k be the smallest

Sheng Qi, Chao Jin, Xuanzhe Liu, and Xin Jin are with the School of Computer Science, Peking University, Beijing 100871, China (e-mail: {shengqi2018, chaojin, liuxuanzhe, xinjinpku}@pku.edu.cn).

Mosharaf Chowdhury is with the Computer Science and Engineering Division, University of Michigan, Michigan 48109-2121, USA (e-mail: mosharaf@umich.edu).

Zhenming Liu is with the Department of Computer Science, College of William & Mary, Virginia 23187-8795, USA (e-mail: zliu@cs.wm.edu).

index s.t. $x_k < 1$. For any $j > k$, we have $x_j = 0$; otherwise k and j form a bad pair. Consequently, we have $x_i = 1$ for $i < k$, and $x_i = 0$ for $i > k$, concluding the proof.

PROOF OF THEOREM IV.2

As described in the main body of the paper, we start by proving the following lemma.

Lemma A.2. *If the step size $\Delta = (\bar{\alpha} - \underline{\alpha})/M$, then $\bar{\alpha} - \underline{\alpha}$ is reduced by a factor of $M/2$ after the current iteration.*

To prove the lemma, we equally divide the current search space $[\underline{\alpha}, \bar{\alpha}]$ into M segments with length Δ . Without loss of generality, assume that our algorithm scans from $\underline{\alpha}$ to $\bar{\alpha}$. Then in the i th step of this iteration, we have $\alpha_{i-1} = \underline{\alpha} + (i-1)\Delta$ and $\alpha_i = \underline{\alpha} + i\Delta$. Also assume that α^* falls within the j th segment, i.e., $[\alpha_{j-1}, \alpha_j]$, which is visited in the j th step, for $j \leq M$. By the unimodality of $R(\alpha)$, we have $R(\alpha_{j-2}) < R(\alpha_{j-1})$ and, therefore, $R'(\alpha_{j-1}) > 0$. However, $R'(\alpha_j)$ can be both positive and negative.

- (i) $R'(\alpha_j) < 0$. Then Algorithm 1 detects a local maxima in line 5. It sets $\bar{\alpha}$ to α_j and $\underline{\alpha}$ to α_{j-2} , ending the current iteration.
- (ii) $R'(\alpha_j) > 0$. Then Algorithm 1 continues to step forward in line 11. Because this step starts from α_j , which is on the right of α^* , we have $R(\alpha_j) > R(\alpha_{j+1})$ and $R'(\alpha_{j+1}) < 0$ by the unimodality of $R(\alpha)$. As a result, the next step of the algorithm falls into the previous case.

We therefore conclude that $\underline{\alpha}$ and $\bar{\alpha}$ are always within two consecutive steps by the end of this iteration, i.e., $\bar{\alpha} - \underline{\alpha} \leq 2\Delta$, which is reduced by a factor of $M/2$ as desired.

Lemma A.2 implies that $\bar{\alpha} - \underline{\alpha}$ falls below δ in at most $O(\log(1/\delta))$ iterations. To prove Theorem IV.2, it remains to show that the number of steps in each iteration is bounded by a constant. This is straightforward since Algorithm 1 essentially performs grid search over the endpoints of M segments, a constant as desired.